

Comprehensive Project Submission: Multi-Tenant Document Intelligence Backend

Developer: Melvin

Project: B2B Multi-Tenant Document Intelligence API

Core Technologies: Python 3.11+, FastAPI, PostgreSQL (pgvector), SQLAlchemy, Redis, Celery, Docker

1. Executive Summary

This document serves as the complete architectural blueprint and technical reference for the Multi-Tenant Document Intelligence Backend. The system is designed as a secure, distributed, microservices-oriented backend that allows enterprise clients to securely upload, organize, and perform AI-driven semantic searches across their proprietary documents.

It mathematically guarantees data privacy between tenants, decouples heavy compute operations to ensure high availability, and integrates advanced vector search capabilities out-of-the-box.

2. Setup & Execution (README)

Prerequisites

- Docker and Docker Compose
- Python 3.11+ (for local development/testing)

Quick Start

1. Environment Configuration:

Copy the template environment file and populate the required secrets:

```
cp .env.example .env
```

Ensure JWT_SECRET_KEY and AI_PROVIDER_API_KEY are securely generated

2. Build and Launch Infrastructure:

The application is fully containerized. Start the API, PostgreSQL, Redis, and Celery worker:

```
docker-compose up --build -d
```

3. Database Migrations:

Execute Alembic migrations to construct the schema once the database is healthy:

```
docker-compose exec api alembic upgrade head
```

4. Accessing the Application:

- **Interactive API Documentation (Swagger):** <http://localhost:8000/docs>
- **Alternative Documentation (ReDoc):** <http://localhost:8000/redoc>

Note: For the required demo, a screencast recording is included in the root directory (demo.mp4), showcasing the User Registration, Document Upload (showing Pending -> Completed async flow), Semantic Search, and Audit Log retrieval.

3. System Architecture & Topology

The architecture separates fast, user-facing synchronous requests from heavy asynchronous compute tasks.

3.1 Component Breakdown

- **API Framework (FastAPI & Pydantic):** Extremely fast ASGI framework for routing. Pydantic acts as a strict validation layer, rejecting malformed payloads before they reach the database.
- **Primary Database (PostgreSQL 15+):** Manages complex relationships. Utilizes the **pgvector** extension to unify relational data and AI vector data, removing the need for a separate vector database (like Pinecone).
- **Message Broker & Cache (Redis):** Acts as the ultra-fast memory bank for Celery queues, rate-limiting (fastapi-limiter), and stateless JWT blocklisting.
- **Background Workforce (Celery):** The invisible workforce that offloads heavy I/O tasks (reading PDFs, chunking text, calling AI APIs) so the main API remains non-blocking.

3.2 Domain-Driven Folder Structure

```
|— app/
|   |— api/      # API Route Controllers (FastAPI Routers)
|   |— core/     # Configuration, Security, JWT, Dependencies
|   |— models/   # SQLAlchemy ORM Data Models
|   |— schemas/  # Pydantic V2 Request/Response Models
|   |— services/ # Business Logic (Document Parser, AI Integration)
|   |— worker/   # Celery Application & Task definitions
|— alembic/     # Database Migration scripts
|— docker-compose.yml # Container orchestration
```

4. Multi-Tenant Data Isolation Strategy

Enforcing strict data boundaries between tenants is the most critical requirement. We utilize a **Shared Database, Shared Schema** topology.

4.1 Isolation Enforcement Mechanism

Tenant isolation is strictly enforced at the framework level, removing reliance on developer

discipline (e.g., manually remembering to write `WHERE company_id = X`):

1. **Identity Extraction:** The `get_current_user` FastAPI dependency decodes the JWT and validates the user.
2. **Context Injection:** The `get_tenant_context` dependency extracts the `company_id`.
3. **Repository Scoping:** This context is injected directly into the SQLAlchemy repository layer. Every single database query implicitly appends `.where(Model.company_id == tenant_id)`. Cross-tenant data retrieval is structurally impossible via the API.

5. Security & Authorization Protocols

- **Authentication:** Passwords are cryptographically hashed using `bcrypt` (via `passlib`). Plaintext is never logged or stored.
- **Stateless JWTs:** Tokens are signed using highly secure symmetric keys. A Redis-backed token blacklist handles secure logout invalidation statelessly.
- **Dynamic Role-Based Access Control (RBAC):** Permissions are enforced dynamically at the routing layer via decorators (e.g., `@require_roles(['Owner', 'Admin'])`). The system validates the exact intersection of the User ID, active Company ID, and required role prior to route execution.
- **Input & File Security:** `Pydantic` drops unknown payload parameters (preventing mass-assignment). File MIME types are validated via `python-magic` to prevent execution of malicious scripts disguised as PDFs, and filenames are sanitized to prevent path traversal.

6. Asynchronous Processing Pipeline (The "Noisy Neighbor" Solution)

To prevent connection timeouts and CPU blocking during large document processing:

1. **Ingestion (Synchronous):** The `/upload` endpoint saves the file, creates a `PENDING` database record, queues a task payload in Redis, and instantly returns `202 Accepted`.
2. **Extraction (Asynchronous):** A Celery worker consumes the task, extracts the text using `PyMuPDF`/`python-docx`, and splits it into semantic chunks (e.g., 500 tokens) to respect LLM limits.
3. **Fault Tolerance:** If the AI provider issues a rate limit, the worker triggers an exponential backoff retry. If text extraction fails completely, the worker safely marks the document status as `FAILED` and releases the thread. The main API remains completely unaffected.

7. Search & AI Integration

- **Keyword Search:** Utilizes PostgreSQL full-text search (`to_tsvector`) for exact phrase matching.
- **Semantic Search:** Documents are chunked and passed to the Gemini API to generate high-dimensional vectors (embeddings). When a user searches, their query is embedded, and the database utilizes the cosine distance operator (`<=>`) via `pgvector` to retrieve the nearest conceptual matches strictly within their `company_id`.

8. Performance Profiling

- **Database Indexing:** B-Tree indexes are applied to `company_id` and `user_id` foreign keys to optimize cross-tenant query speeds.
- **Vector Optimization:** An **HNSW (Hierarchical Navigable Small World)** index is applied to the embedding column in `pgvector` to ensure sub-50ms semantic query times even at scale.

9. Known Limitations & Production Improvements

While robust for this assessment, the following upgrades are necessary for an enterprise production deployment:

1. **Storage Architecture Upgrade (S3 + Pre-Signed URLs):** * *Limitation:* Storing uploaded documents within Docker container volumes consumes backend bandwidth and limits scaling.
 - *Improvement:* Transition to AWS S3. The FastAPI server generates a cryptographically signed Pre-Signed URL. The client uploads the file *directly* to S3, bypassing our API entirely.
2. **Database Row-Level Security (RLS):** * *Limitation:* Application-level isolation relies on the ORM routing correctly.
 - *Improvement:* Implement PostgreSQL RLS policies. This pushes the security boundary down to the database kernel itself (e.g., `CREATE POLICY tenant_isolation ON documents USING (company_id = current_setting('app.current_tenant')::int);`).
3. **Scaling to 1 Million Users:**
 - Transition from Docker Compose to **Kubernetes (K8s)**.
 - Implement **KEDA** to dynamically auto-scale Celery worker pods based on Redis queue depth.
 - Split PostgreSQL into a Primary (Write) instance and multiple Read-Replicas via **PgBouncer**, routing semantic searches strictly to the replicas.

10. Live Change Task: Secure Audit Log Retrieval

Task Requirement: Add an endpoint to return the last 10 audit log entries for a company. Only Owner and Admin can access it.

Implementation:

Utilizing the architectural patterns established, this endpoint securely injects the database session, validates identity, enforces RBAC, and scopes the query to the tenant.

```
from fastapi import APIRouter, Depends, Request
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, desc
from typing import List
```

```
from app.core.dependencies import get_db_session, get_current_user, require_role
from app.models.audit import AuditLog
```

```

from app.schemas.audit import AuditLogResponse
from app.models.user import User

router = APIRouter(prefix="/api/v1/audit", tags=["Audit & Compliance"])

@router.get("/logs", response_model=List[AuditLogResponse])
async def get_recent_audit_logs(
    request: Request,
    db: AsyncSession = Depends(get_db_session),
    current_user: User = Depends(get_current_user),
    # The dependency below ensures the request is halted with HTTP 403 if unauthorized
    _authorized: bool = Depends(require_role(allowed_roles=["Owner", "Admin"])),
):
    """
    Retrieves the 10 most recent audit logs for the authenticated user's company.
    Strictly isolated by the company_id embedded in the verified JWT context.
    """

    # Architecture Note: current_user.company_id is validated during JWT decoding
    query = (
        select(AuditLog)
        .where(AuditLog.company_id == current_user.company_id) # Strict Tenant Isolation
        .order_by(desc(AuditLog.timestamp))
        .limit(10)
    )

    result = await db.execute(query)
    logs = result.scalars().all()

    return logs

```